ИТМО

Факультет Программной Инженерии и Компьютерной техники

МОДУЛЬ 2. ОБЩИЙ ОТЧЁТ ПО ЛАБОРАТОРНЫМ РАБОТАМ №3–№6: МЕТОДЫ ОБУЧЕНИЯ С УЧИТЕЛЕМ

Выполнили

Ануфриев А.С. 465029

Проверил

Болдырева Е. А.

Аннотация – В отчёте обобщены результаты четырёх лабораторных работ модуля 2 по линейной регрессии, методу k-ближайших соседей (k-NN), деревьям решений и логистической регрессии. Все модели реализованы “с нуля” на языке Python с использованием только NumPy и Pandas без готовых библиотек машинного обучения. Для каждой работы приведено описание метода, псевдокод, результаты экспериментов на реальных датасетах (жильё Калифорнии, вино, грибы UCI, пассажиры Титаника), графики и выводы. В заключении дан сравнительный анализ методов: их особенности, преимущества и ограничения. В приложении приводятся фрагменты исходного кода реализации.

Ключевые слова: линейная регрессия, k-NN, дерево решений, логистическая регрессия, NumPy, Python, классификация, регрессия

08.11.2026

Содержание

1 Введение **2**

2 Лабораторная работа 1. Линейная регрессия (данные о жилье Калифорнии) 2
2.1 Цели и задачи . . . 2
2.2 Описание метода . . . 3
2.3 Псевдокод метода . . . 3
2.4 Исходные данные . . . 3
2.5 Ход работы . . . 3
2.6 Выводы . . . 5

3 Лабораторная работа 2. Метод k-ближайших соседей (датасет о вине) 6
3.1 Цели и задачи . . . 6
3.2 Описание метода . . . 6
3.3 Псевдокод метода . . . 6
3.4 Исходные данные . . . 6
3.5 Ход работы . . . 7
3.6 Выводы . . . 8

4 Лабораторная работа 3. Деревья решений (датасет грибов UCI Mushroom) 8
4.1 Цели и задачи . . . 8
4.2 Описание метода . . . 8
4.3 Псевдокод метода . . . 9
4.4 Исходные данные . . . 9
4.5 Ход работы . . . 9
4.6 Выводы . . . 10

5 Лабораторная работа 4. Логистическая регрессия (датасет пассажиров Титаника) 10
5.1 Цели и задачи . . . 10
5.2 Описание метода . . . 11
5.3 Псевдокод метода . . . 11
5.4 Исходные данные и предобработка . . . 12
5.5 Исследование гиперпараметров . . . 13
5.6 Выводы . . . 14

6 Сравнительный анализ методов 15
6.1 Сравнительная таблица . . . 15
6.2 Обсуждение . . . 15
6.3 Примеры лучшего применения каждого метода . . . 16

7 Заключение 16

8 Приложение. Исходный код 16
8.1 Линейная регрессия (лаб. 3, `lab3_regression.py`) . . . 16
8.2 k-ближайших соседей (лаб. 4, `knn_wine.py`) . . . 17
8.3 Дерево решений (лаб. 5, `lab5_tree.py`) . . . 17
8.4 Логистическая регрессия (лаб. 6, `lab6_logreg.py`) . . . 19

1 Введение

Модуль 2 посвящён основным методам (supervised) обучения с учителем: регрессии и классификации. В ходе четырёх лабораторных работ (лабораторные №3–№6) были реализованы и исследованы четыре метода:

1. линейная регрессия методом наименьших квадратов (лабораторная №3);
2. метод k-ближайших соседей k-NN (лабораторная №4);
3. дерево решений (лабораторная №5);
4. логистическая регрессия (лабораторная №6).

Общая особенность всех работ — реализация алгоритмов собственным кодом на Python с использованием только NumPy и Pandas. Готовые средства машинного обучения (sklearn и аналоги) не применялись. В каждой работе проведена загрузка и предобработка реального датасета, разделение на обучающую и тестовую выборки, обучение модели, оценка качества и оформление результатов в виде отдельного отчёта с графиками. Ниже по каждой лабораторной работе приведены цель, описание метода, псевдокод, результаты и выводы, а затем — сравнительный анализ всех четырёх методов.

2 Лабораторная работа 1. Линейная регрессия (данные о жилье Калифорнии)

2.1 Цели и задачи

Цель работы — построение и сравнение моделей линейной регрессии для прогнозирования стоимости жилья в Калифорнии методом наименьших квадратов без применения готовых библиотек регрессии. Основные задачи:

1. получить и визуализировать описательную статистику по датасету (количество, среднее, стандартное отклонение, минимум, максимум, квантили);
2. выполнить предобработку данных: обработку отсутствующих значений, кодирование категориальных признаков, нормировку;
3. разделить данные на обучающую и тестовую выборки;
4. реализовать линейную регрессию методом наименьших квадратов собственным кодом (минимизация суммы квадратов разностей фактических и предсказанных значений), используя только NumPy и Pandas;
5. построить три модели с разными наборами признаков;
6. оценить каждую модель коэффициентом детерминации R^2 и сравнить результаты;
7. в качестве бонуса ввести синтетический признак.

2.2 Описание метода

Линейная множественная регрессия описывает зависимость результативного признака y от нескольких факторов $x_1, \ldots, x_p$:

$$\hat{y} = a + b_1 x_1 + \cdots + b_p x_p, \tag{1}$$

где a — свободный член, b_j — регрессионные коэффициенты. Метод наименьших квадратов (МНК) определяет оценки параметров вектором $B = (a, b_1, \ldots, b_p)^\top$ как

$$B = (X^\top X)^{-1} X^\top Y, \tag{2}$$

где X — матрица признаков с первым столбцом единиц (свободный член), Y — вектор наблюдений целевой переменной. Качество модели оценивается коэффициентом детерминации

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}. \tag{3}$$

2.3 Псевдокод метода

```
# X: feature matrix, y: target vector
# add column of ones for the intercept term
X_aug = column_stack([ones(len(X)), X])
# OLS coefficients: B = (X^T X)^{-1} X^T y
B = inv(X_aug.T @ X_aug) @ X_aug.T @ y
# prediction
y_pred = X_aug_test @ B
# quality: coefficient of determination
R2 = 1 - sum((y_true - y_pred)^2) / sum((y_true - mean(y_true))^2)
```

2.4 Исходные данные

Исходный датасет содержит 17 000 записей о жилых районах Калифорнии. Все 9 признаков числовые, пропущенных значений и категориальных признаков в исходных данных отсутствуют. Целевая переменная — *median_ house_ value* (медианная стоимость дома, тыс. долл.). Особенность данных: значения стоимости ограничены сверху отметкой 500 000 (цензурное отсечение); таких записей 22 (0,13 %), доход у них разный, поэтому это не выброс, подлежащий удалению, а техническая особенность датасета.

2.5 Ход работы

Описательная статистика. Вычислены размер выборки, среднее, стандартное отклонение, минимум, максимум и квантили 25%, 50%, 75% для всех признаков. Для целевой переменной построена гистограмма распределения, для пары “доход населения — стоимость жилья” — диаграмма рассеяния (рисунки 1, 2).

Таблица 1: Описательная статистика по датасету о жилье Калифорнии.

Признак	count	mean	std	min	25%	50%	75%	max
longitude	17000	-119.56	2.01	-124.35	-121.79	-118.49	-118.00	-114.31
latitude	17000	35.63	2.14	32.54	33.93	34.25	37.72	41.95
housing_median_age	17000	28.59	12.59	1.0	18.0	29.0	37.0	52.0
total_rooms	17000	2643.66	2179.95	2.0	1462.0	2127.0	3151.25	37937.0
total_bedrooms	17000	539.41	421.50	1.0	297.0	434.0	648.25	6445.0
population	17000	1429.57	1147.85	3.0	790.0	1167.0	1721.0	35682.0
households	17000	501.22	384.52	1.0	282.0	409.0	605.25	6082.0
median_income	17000	3.88	1.91	0.50	2.57	3.54	4.77	15.00
median_house_value	17000	207300.91	115983.76	14999.0	119400.0	180400.0	265000.0	500001.0

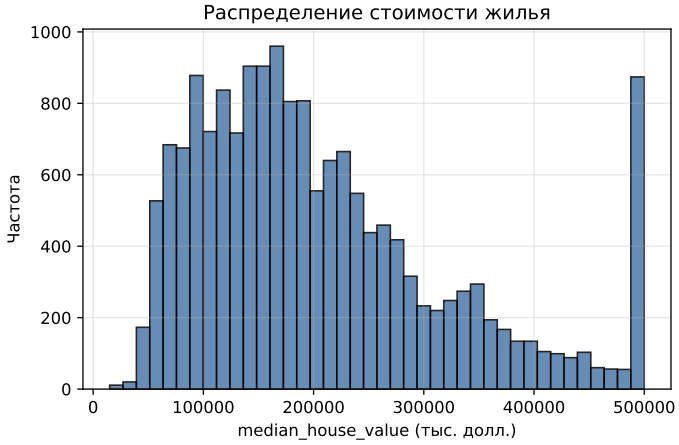


Рис. 1: Гистограмма распределения медианной стоимости жилья (лаб. 3).

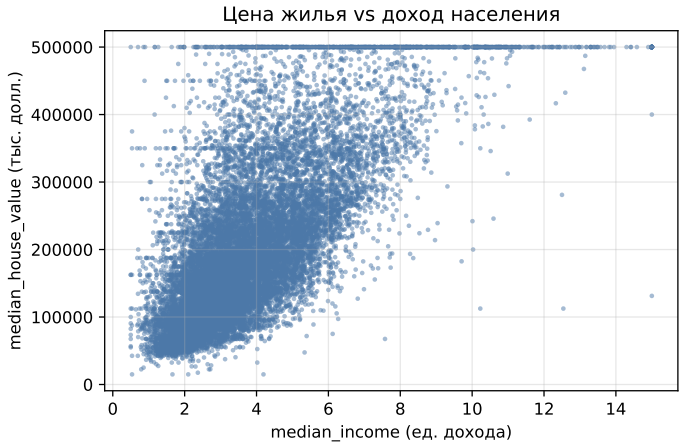


Рис. 2: Диаграмма рассеяния стоимости жилья от дохода населения (лаб. 3).

Предобработка. Пропущенных значений не обнаружено, категориальных признаков нет. Все числовые признаки нормированы стандартизацией (вычет среднего, деление на стандартное отклонение); целевая переменная не нормировалась.

Разделение на выборки. Данные перемешаны и разделены на обучающую (13 600 записей, 80%) и тестовую (3 400 записей, 20%) с фиксированным зерном генератора random_state = 42.

Реализация. Коэффициенты регрессии вычислены собственным кодом на Python с использованием NumPy по формуле $B = (X^\top X)^{-1} X^\top Y$, что эквивалентно минимизации суммы квадратов разностей фактических и предсказанных значений. Готовые средства регрессии не использовались.

Три модели. Построены три модели с разными наборами признаков. В третью модель включён синтетический признак $rooms_per_household = total_rooms / households$ (бонусное задание).

Таблица 2: Сравнение трёх моделей линейной регрессии (признаки нормированы).

Модель	Признаки	R^2 на тестовой выборке
Модель 1	median_income	0.4978
Модель 2	median_income, housing_median_age	0.5376
Модель 3	median_income, total_rooms, households, rooms_per_household	0.5227

Таблица 3: Оценочные коэффициенты моделей (включая свободный член, признаки нормированы).

	Модель 1	Модель 2	Модель 3
Свободный член	207066.73	206962.89	207033.39
median_income	79512.76	81929.71	89613.01
housing_median_age	—	21519.53	—
total_rooms	—	—	-54374.09
households	—	—	56290.68
rooms_per_household	—	—	-6.13

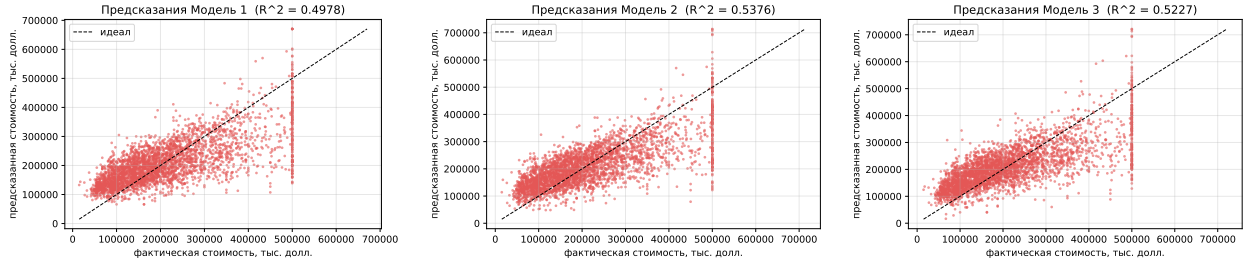


Рис. 3: Фактические против предсказанных значений на тестовой выборке: Модель 1, 2, 3 (лаб. 3).

2.6 Выводы

Наилучшее качество показала Модель 2 (доход населения и возраст жилого дома), $R^2 \approx 0.54$ на тестовой выборке. Доход населения — сильнейший отдельный признак ($R^2 \approx 0.50$ в одномерной модели). Добавление возраста дома повысило качество, а в третью модель количество комнат, число домохозяйств и синтетический признак не улучшили результат ($R^2 \approx 0.52$) из-за сильной линейной зависимости (мультиколлинеарности) между этими признаками.

3 Лабораторная работа 2. Метод k-ближайших соседей (датасет о вине)

3.1 Цели и задачи

Цель работы — исследование и применение метода k-ближайших соседей (k-NN) для классификации датасета о вине. Основные задачи:

1. описательная статистика и визуализация признаков (включая 3D-визуализацию);
2. предобработка данных: пропуски, кодирование категориальных признаков, масштабирование;
3. реализация k-NN собственным кодом без готовых библиотек классификации;
4. постройка двух моделей: со случайным и с фиксированным наборами признаков;
5. оценка качества на тестовой выборке при различных k и построение матриц ошибок.

3.2 Описание метода

Метод k-ближайших соседей относит объект к классу, наиболее часто встречающемуся среди k ближайших соседей в пространстве признаков. Расстояние вычисляется по правилу Евклида:

$$d(\mathbf{x}_i, \mathbf{x}_j) = \sqrt{\sum_{p=1}^{P} (x_{ip} - x_{jp})^2}. \tag{4}$$

Класс определяется majoritaire голосованием среди k ближайших обучающих примеров. Качество оценивается долей верных ответов (accuracy) и матрицей ошибок. k-NN — "ленивый" алгоритм: этапа обучения в классическом смысле нет, все вычисления выполняются на этапе предсказания.

3.3 Псевдокод метода

```
# X_train, y_train: training set; X: test objects
# pairwise Euclidean distances: ||a-b||^2 = ||a||^2 + ||b||^2 - 2 a*b
d = sqrt(sum(X^2, axis=1) + sum(X_train^2, axis=1).T - 2 * X @ X_train.T)
# k nearest neighbours for each test object
nearest = argsort(d, axis=1)[:, :k]
# class: majority vote among neighbour labels
y_pred[i] = argmax(bincount(y_train[nearest[i]]))
```

3.4 Исходные данные

Исходный датасет содержит 178 записей о вине. Признаков: 13 числовых (Alcohol, Malic Acid, Ash, Alcalinity of ash, Magnesium, Total phenols, Flavanoids, Nonflavanoid phenols, Proanthocyanins, Color intensity, Hue, OD280/OD315 of diluted wines, Proline). Целевая переменная — Wine (три класса: 1, 2, 3). Пропущенных значений и категориальных признаков не обнаружено.

3.5 Ход работы

Предобработка. Все 13 числовых признаков подвергнуты стандартизации. Данные разделены на обучающую (142 записи, 80%) и тестовую (36 записей, 20%) с random_state = 42.

Реализация. k-NN реализован на NumPy: парные евклидовые расстояния вычисляются матрично, затем сортировка и выбор k ближайших соседей, класс определяется голосованием. Готовые средства классификации не использовались.

Модели. Построены две модели:

- Модель 1 — признаки отбираются случайно (5 из 13: Alcohol, Color intensity, OD280/OD315 of diluted wines, Proanthocyanins, Total phenols);
- Модель 2 — фиксированный набор признаков (Alcohol, Malic Acid, Ash, Alcalinity of ash, Magnesium, Total phenols).

Оценка при $k = 3, 5, 10, 15$.

Таблица 4: Точность k-NN на тестовой выборке.

Модель	Признаки	k=3	k=5	k=10	k=15
Модель 1	случайные (5 признаков)	0.9444	0.9444	0.9167	0.9167
Модель 2	фиксированные (6 признаков)	0.8889	0.8889	0.9444	0.9444

Таблица 5: Матрица ошибок, Модель 1, k=5 (строки — истинные классы, столбцы — предсказанные).

	Предсказанный класс		
Истинный класс	1	2	3
1	14	0	0
2	1	12	1
3	0	0	8

Таблица 6: Матрица ошибок, Модель 2, k=10.

	Предсказанный класс		
Истинный класс	1	2	3
1	13	0	1
2	0	14	0
3	0	1	7

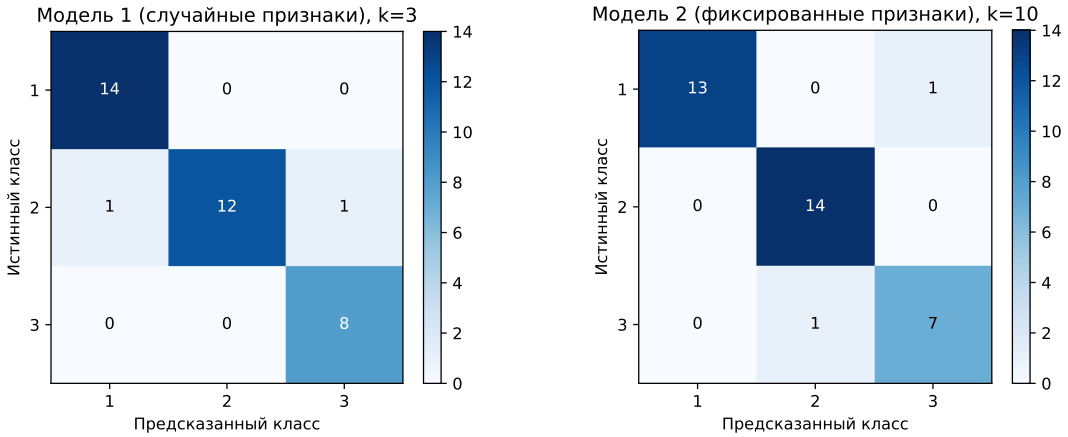


Рис. 4: Матрицы ошибок: Модель 1 (слева, k=5) и Модель 2 (справа, k=10) (лаб. 4).

3.6 Выводы

Обе модели обеспечивают высокую точность на тестовой выборке (до 0.9444 из 36 объектов). Модель 1 стабильна при $k = 3$ и $k = 5$ (0.9444), Модель 2 достигает наилучшего результата при $k = 10$ и $k = 15$ (0.9444). Основные ошибки приходятся на путаницу между классами 2 и 3, которые ближе друг к другу в пространстве признаков.

4 Лабораторная работа 3. Деревья решений (датасет грибов UCI Mushroom)

4.1 Цели и задачи

Целью работы является реализация алгоритма построения дерева решений для задачи бинарной классификации грибов (съедобный / ядовитый) и оценка его качества. Основные этапы:

1. загрузка и предобработка исходных данных;
2. случайный отбор $\sqrt{n}$ признаков;
3. построение небинарного дерева решений собственным кодом без сторонних библиотек для построения дерева;
4. оценка по Accuracy, precision и recall без использования библиотек;
5. построение кривых AUC-ROC и AUC-PR без использования библиотек.

4.2 Описание метода

Дерево решений — иерархическая структура правил классификации: каждый внутренний узел содержит проверку по какому-то признаку, ребра соответствуют возможным значениям признака, а листья содержат метки классов. Построение выполняется рекурсивно сверху вниз (подход алгоритма C4.5). В каждом узле выбирается признак с максимальным нормированным приростом информации:

$$\text{GainRatio}(X) = \frac{\text{Info}(T) - \text{Info}_X(T)}{\text{splitinfo}(X)}, \tag{5}$$

где $\mathrm{Info}(T) = -\sum_j p_j \log_2 p_j$ — энтропия множества примеров, $\mathrm{Info}_X(T) = \sum_i \frac{|T_i|}{|T|}\mathrm{Info}(T_i)$ — условная энтропия после разбиения по признаку X, $\mathrm{splitinfo}(X) = -\sum_i \frac{|T_i|}{|T|}\log_2\frac{|T_i|}{|T|}$. По выбранному признаку создаётся столько потомков, сколько у него значений (дерево не бинарное). Лист получает класс большинства объектов в узле.

4.3 Псевдокод метода

```
build_tree(X, y, features, depth):
    node = majority class of y; share of poisonous = mean(y == 1)
    if all examples one class or depth = max or no features left:
        node.is_leaf = True; return node
    best feature = argmax information gain (GainRatio) over features
    if no gain: node.is_leaf = True; return node
    for each value v of best feature:
        child[v] = build_tree(X[feature==v], y[feature==v], features without best, depth+1)
    return node

predict(node, object):
    descend to leaf; return label or share of poisonous (probability)
```

4.4 Исходные данные

Используется датасет UCI Mushroom (*agaricus-lepiota.data*, семейства *Agaricus* и *Lepiota*, источник — справочник *The Audubon Society Field Guide to North American Mushrooms*, 1981). Датасет содержит 8124 объекта, каждый описывается классом (съедобный `e` / ядовитый `p`) и 22 номинативными признаками; пропуски *?* встречаются только в признаке *stalk-root*. После удаления объектов с пропусками остаётся 5644 записи: 3488 съедобных (61.8%) и 2156 ядовитых (38.2%).

4.5 Ход работы

1. Чтение данных; исключение записей с пропусками в *stalk-root*.
2. Случайный отбор $\sqrt{22} \approx 4$ признаков с зерном 42: *cap-shape*, *cap-surface*, *gill-color*, *stalk-color-above-ring*.
3. Стратифицированное разделение 70/30: обучение 3951 объект, тест 1693 объекта, соотношение классов сохранено.
4. Построение дерева на обучении: максимальная глубина 12, фактическая глубина 4, 85 внутренних узлов, 63 листа.
5. Метки классов и вероятности (доля ядовитых в листе) для теста.
6. Accuracy, precision, recall, AUC-ROC, AUC-PR собственным кодом (метод трапеций для площадей под кривыми).

Таблица 7: Матрица ошибок и метрики качества на тестовой выборке (1693 объекта, положительный класс — ядовитый гриб).

Значение			
TP (яды распознаны)	472	FP (съедобные как яды)	71
TN (съедобные распознаны)	975	FN (яды как съедобные)	175
Accuracy			0.8547
Precision			0.8692
Recall			0.7295
AUC-ROC			0.9416
AUC-PR			0.9223

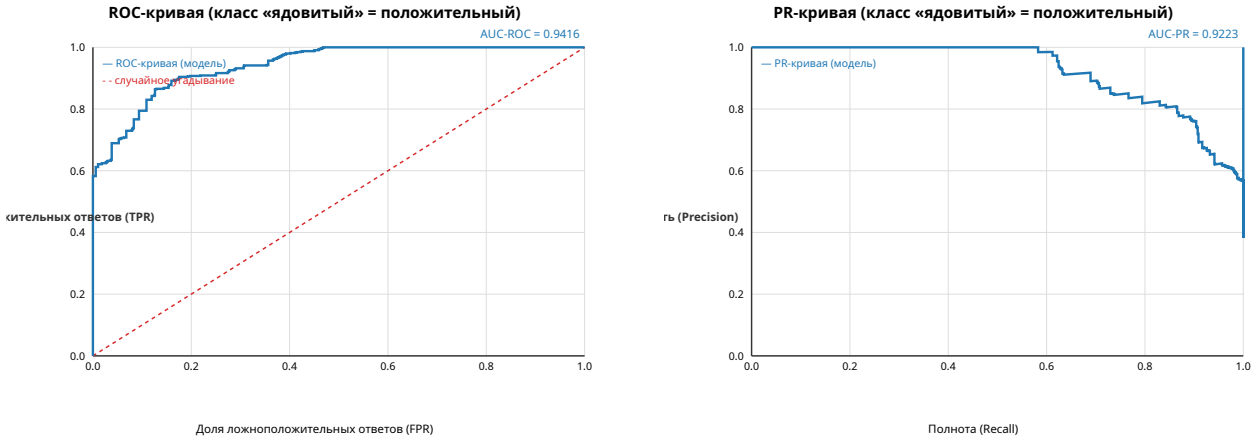


Рис. 5: ROC- и PR-кривые классификатора на тестовой выборке (лаб. 5): AUC-ROC = 0.9416, AUC-PR = 0.9223.

4.6 Выводы

Реализовано небинарное дерево решений без сторонних библиотек. На 4 случайно отобранных признаках дерево глубины 4 достигает Accuracy 0.8547, precision 0.8692 и recall 0.7295; AUC-ROC = 0.9416, AUC-PR = 0.9223. Получение вероятностей из долей объектов в листьях позволяет построить ROC- и PR-кривые. Ограничение: recall 0.7295 означает, что часть ядовитых грибов классифицируется как съедобные, что критично для задачи безопасного потребления.

5 Лабораторная работа 4. Логистическая регрессия (датасет пассажиров Титаника)

5.1 Цели и задачи

Целью работы является реализация модели логистической регрессии для бинарной классификации без использования сторонних библиотек машинного обучения, исследование влияния гиперпараметро на качество модели и оценка её работы на тестовых данных.

1. загрузка и предобработка датасета пассажиров Титаника;
2. описательная статистика и её визуализация;

3. разделение на обучающую и тестовую выборки;

4. реализация логистической регрессии “с нуля” (сигмоида, log-loss, градиентный спуск, настраиваемая скорость обучения и число итераций);

5. исследование гиперпараметров: скорость обучения, число итераций, метод оптимизации (градиентный спуск и оптимизация Ньютона);

6. оценка по accuracy, precision, recall, F1-Score на тестовой выборке.

5.2 Описание метода

Логистическая регрессия моделирует вероятность принадлежности объекта к положительному классу с помощью сигмоидной функции:

$$\sigma(z) = \frac{1}{1+e^{-z}}, \qquad p = \sigma(Xw), \tag{6}$$

где X — матрица признаков со столбцом смещения, w — вектор весов, p — предсказанная вероятность. Порог классификации обычно равен 0.5: $p \geq 0.5 \Rightarrow$ класс 1.

Функция потерь (логарифмические потери, кросс-энтропия):

$$L(w) = -\frac{1}{N}\sum_{i=1}^{N} \left[y_i \ln p_i + (1-y_i)\ln(1-p_i) \right]. \tag{7}$$

Обучение — градиентный спуск:

$$w \leftarrow w - \frac{\eta}{N} X^\top (p - y), \tag{8}$$

где η — скорость обучения. Для оптимизации Ньютона используется гессиан $H = \frac{1}{N} X^\top W X$, $W = \mathrm{diag}(p_i(1-p_i))$, и обновление $w \leftarrow w - H^{-1}\nabla L(w)$. Функция выпукла, поэтому градиентные методы сходятся к глобальному минимуму.

5.3 Псевдокод метода

```
sigmoid(z) = 1 / (1 + exp(-clip(z, -500, 500)))
log_loss(y, p) = -mean(y*log(p) + (1-y)*log(1-p))

gradient_descent(X, y, lr, n_iter):
    w = zeros
    repeat n_iter times:
        p = sigmoid(X @ w)
        grad = X.T @ (p - y) / N
        w = w - lr * grad

Newton(X, y, n_iter):
    repeat n_iter times:
        p = sigmoid(X @ w)
        grad = X.T @ (p - y) / N
        W = diag(p * (1 - p))
        H = X.T @ W @ X / N
        w = w - solve(H, grad)

predict(X, w): return (sigmoid(X @ w) >= 0.5)
```

5.4 Исходные данные и предобработка

Используется датасет пассажиров Титаника, 891 запись. Целевой признак — *Survived* (0 — не выжил, 1 — выжил); доля выживших около 38,4%. Числовые признаки: *Pclass*, *Age*, *SibSp*, *Parch*, *Fare*; категориальные: *Sex*, *Embarked*. Признаки *Name*, *Ticket*, *Cabin*, *PassengerId* исключены. Пропуски: в *Age* — 177, в *Embarked* — 2, в *Cabin* — 687 (признак исключён).

Предобработка без утечки информации:

1. данные разделены на обучающую и тестовую 80/20 (712 / 179, зерно 42) *до* любой предобработки;
2. статистики для заполнения пропусков и стандартизации вычислены только на обучающей выборке (медиана Age/Fare, мода Embarked, среднее и std числовых признаков);
3. те же статистики применены к тестовой выборке;
4. пропуски Age и Fare заполнены медианой обучения, Embarked — модой обучения;
5. Sex закодирован (male = 0, female = 1), Embarked — one-hot кодированием (порты S, C, Q);
6. числовые признаки стандартизированы статистиками обучения.

После предобработки в модели используется 9 признаков (5 числовых + пол + 3 порта one-hot).

Таблица 8: Описательная статистика по датасету Титаника (все 891 запись после заполнения пропусков).

Признак	Count	Mean	Std	Min	25%	50%	75%	Max
Survived	891	0.3838	0.4866	0.0	0.0	0.0	1.0	1.0
Pclass	891	2.3086	0.8361	1.0	2.0	3.0	3.0	3.0
Age	891	29.3616	13.0197	0.42	22.0	28.0	35.0	80.0
SibSp	891	0.5230	1.1027	0.0	0.0	0.0	1.0	8.0
Parch	891	0.3816	0.8061	0.0	0.0	0.0	0.0	6.0
Fare	891	32.2042	49.6934	0.0	7.91	14.45	31.0	512.33

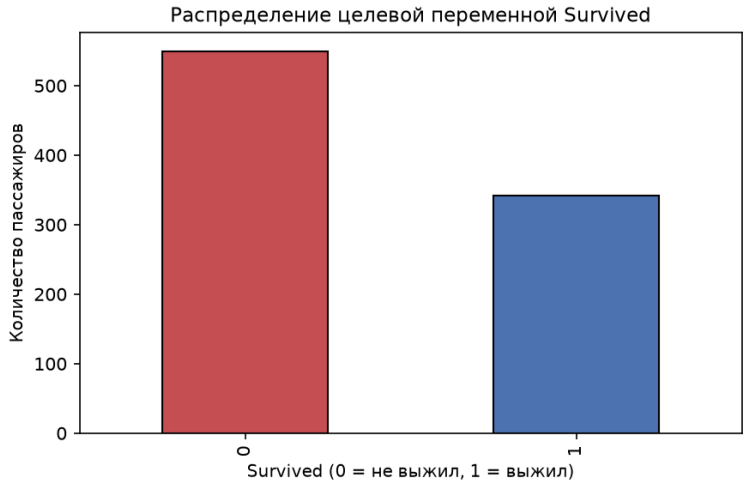


Рис. 6: Распределение целевой переменной Survived (лаб. 6).

5.5 Исследование гиперпараметров

Проведено исследование: три значения скорости обучения (0.01, 0.1, 0.5), три числа итераций (100, 500, 1000) и два метода оптимизации (градиентный спуск, оптимизация Ньютона).

Таблица 9: Качество модели на тестовой выборке для комбинаций гиперпараметров.

Метод	lr	Iter.	Loss	Acc	Prec.	Rec.	F1
GD	0.01	100	0.6340	0.6983	0.6341	0.4000	0.4906
GD	0.01	500	0.5490	0.7207	0.6744	0.4462	0.5370
GD	0.01	1000	0.5115	0.7430	0.6863	0.5385	0.6034
GD	0.1	100	0.5118	0.7430	0.6863	0.5385	0.6034
GD	0.1	500	0.4503	0.7765	0.6866	0.7077	0.6970
GD	0.1	1000	0.4449	0.7765	0.6866	0.7077	0.6970
GD	0.5	100	0.4503	0.7765	0.6866	0.7077	0.6970
GD	0.5	500	0.4442	0.7765	0.6866	0.7077	0.6970
GD	0.5	1000	0.4442	0.7765	0.6866	0.7077	0.6970
Newton	—	100	0.4442	0.7765	0.6866	0.7077	0.6970
Newton	—	500	0.4442	0.7765	0.6866	0.7077	0.6970
Newton	—	1000	0.4442	0.7765	0.6866	0.7077	0.6970

Наилучший результат по F1-Score показывает градиентный спуск с learning rate 0.1 и 500 итерациями: accuracy 0.7765, precision 0.6866, recall 0.7077, F1-Score 0.6970 (TP = 46, TN = 93, FP = 21, FN = 19). Увеличение learning rate свыше 0.1 и числа итераций свыше 500 не улучшает метрики: loss стабилизируется. Метод Ньютона сходится за немногие итерации, но не даёт преимущества в точности на тесте по сравнению с градиентным спуском.

Таблица 10: Матрица ошибок лучшей модели (GD, lr = 0.1, 500 итераций) на тестовой выборке.

	Предсказано: не выжил	Предсказано: выжил
На самом деле: не выжил	TN = 93	FP = 21
На самом деле: выжил	FN = 19	TP = 46

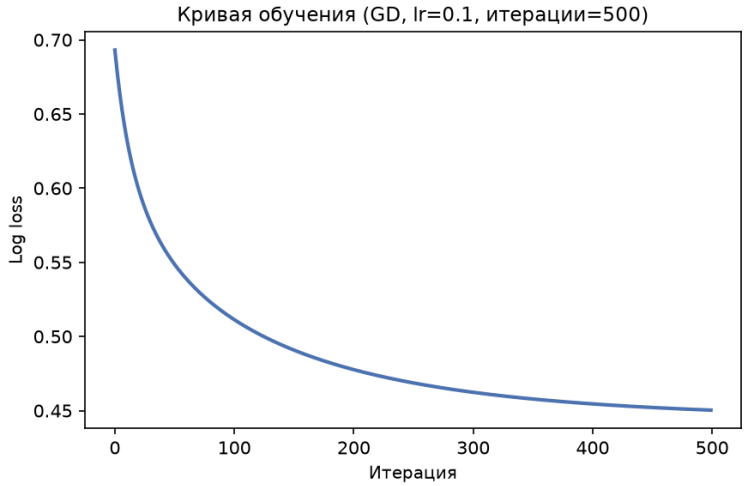


Рис. 7: Кривая обучения (градиентный спуск, lr = 0.1, 500 итераций) (лаб. 6).

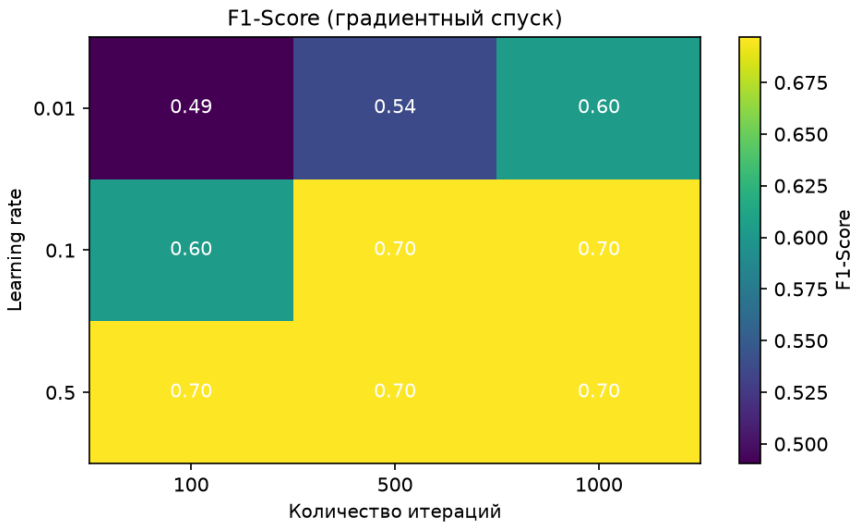


Рис. 8: Зависимость F1-Score от learning rate и числа итераций (градиентный спуск) (лаб. 6).

5.6 Выводы

Реализована логистическая регрессия “с нуля” на Python (NumPy/Pandas): сигмоида, log-loss, градиентный спуск с настраиваемой скоростью обучения и числом итераций, а также оптимизация Ньютона. Проведена предобработка без утечки данных. Категориальный признак Embarked zakодирован one-hot кодированием. Наилучшее качество по F1-Score (0.6970) достигается при learning rate 0.1, 500 итерациях и градиентном спуске. Классы несбалансированы (доля выживших 38,4%), что снижает качество предсказания меньшего класса при общем высоком accuracy.

6 Сравнительный анализ методов

6.1 Сравнительная таблица

Таблица 11: Сравнение четырёх методов модуля 2.

Метод	Тип задачи	Датасет, качество (тест)	Преимущества	Ограничения
Линейная регрессия (МНК)	Регрессия (непрерывная цель)	Жильё Калифорнии, R^2 до 0.54	Простота, интерпретируемость коэффициентов, аналитическое решение, быстрота	Линейная зависимость, чувствительность к мультиколлинеарности и выбросам
k-ближайших соседей	Классификация (и регрессия)	Вино, accuracy до 0.9444	Простота, нет этапа обучения, непараметричность, работает с любым числом классов	Медленное предсказание, "проклятие размерности", чувствительность к масштабу признаков и выбору k
Дерево решений	Классификация	Грибы UCI, accuracy 0.8547, AUC-ROC 0.9416	Интерпретируемость, номинативные признаки, нелинейные границы, мало подготовки данных	Склонность к переобучению, нестабильность, жадный алгоритм
Логистическая регрессия	Бинарная классификация	Титаник, accuracy 0.7765, F1 0.6970	Вероятностный вывод, быстрая сходимость, хорошая база	Линейная граница разделения, сложности с нелинейными зависимостями

6.2 Обсуждение

Все четыре метода показали приемлемое качество на своих задачах. Линейная регрессия и логистическая регрессия — линейные модели: первые предсказывают непрерывное значение через среднеквадратичную функцию потерь, вторые через сигмоиду и кросс-энтропию выдают вероятность класса. k-NN и дерево решений — непараметрические методы, capables capturing нелинейных зависимостей без явной функциональной формы.

k-NN не требует обучения, но стоимость предсказания растет с размером выборки; качество сильно зависит от масштабирования признаков (в работе без стандартизации признаки с большими значениями, например Proline 747, доминировали бы в расстоянии). Дерево решений естественно работает с номинативными признаками и даёт легко интерпретируемые правила, но может переобучаться (в работе ограничение глубины и выбор $\sqrt{n}$ признаков служили регуляризацией). Логистическая регрессия даёт калиброванные вероятности и работает лучше всего при линейно разделимых классах; при сильной нелинейности или большом числе взаимодействий признаков уступает деревьям и k-NN.

6.3 Примеры лучшего применения каждого метода

- **Линейная регрессия:** прогнозирование числовых величин, когда связь приближённо линейна (стоимость недвижимости, спрос, продажи), когда нужна интерпретируемость влияния каждого фактора.
- **k-NN:** задачи классификации на небольших и средних выборках с осмысленной метрикой расстояния, системы рекомендаций, задачи без четкой функциональной зависимости между признаками и целевой переменной.
- **Деревья решений:** смешанные числовые и категориальные данные, задачи, важна интерпретируемость решений (кредитный скоринг, медицинская диагностика), когда не требуется строгая параметрическая форма зависимости.
- **Логистическая регрессия:** бинарная классификация с вероятностным выводом (отток клиентов, скоринг рисков, выживание), особенно как базовая модель, которую легко обучать и интерпретировать.

7 Заключение

В рамках модуля 2 реализованы четыре метода надзоруемого обучения — линейная регрессия, k-NN, дерево решений и логистическая регрессия — полностью собственным кодом на Python (NumPy/Pandas) без готовых библиотек машинного обучения. По каждому методу получены результаты на реальных датасетах: $R^2 \approx 0.54$ для регрессии стоимости жилья, accuracy до 0.9444 при k-NN на данных о вине, accuracy 0.8547 с AUC-ROC 0.9416 при классификации грибов, accuracy 0.7765 и F1 0.6970 для логистической регрессии на датасете Титаника. Линейные модели просты и интерпретируемы, но предполагают линейную зависимость; непараметрические методы (k-NN, деревья) лучше улавливают нелинейные зависимости, но требуют тщательной предобработки и защиты от переобучения. Выбор метода зависит от природы задачи (регрессия против классификации), типа признаков, объема данных и требований к интерпретируемости.

8 Приложение. Исходный код

Ниже приводятся ключевые фрагменты реализации каждого метода (полные скрипты хранятся в папках `prog/` соответствующих лабораторных работ).

8.1 Линейная регрессия (лаб. 3, `lab3_regression.py`)

```
def preprocess(df):
    # check missing values, encode categorical features,
    # standardize numeric features (target not normalized)
    report = {}
    report["missing_total"] = int(df.isna().sum().sum())
    categorical_cols = df.select_dtypes(exclude=[np.number]).columns.tolist()
    ...
    feature_cols = [c for c in df.columns if c != TARGET]
    means = df[feature_cols].mean()
    stds = df[feature_cols].std().replace(0, 1.0)
    df[feature_cols] = (df[feature_cols] - means) / stds
    return df, report

def ols_fit(X, y):
```

```
    # B = (X^T X)^{-1} X^T y, first column of ones added here
    X_aug = np.column_stack([np.ones(len(X)), X])
    XtX = X_aug.T @ X_aug
    XtX_inv = np.linalg.inv(XtX)
    B = XtX_inv @ X_aug.T @ y
    return B

def ols_predict(X, B):
    X_aug = np.column_stack([np.ones(len(X)), X])
    return X_aug @ B

def r2_score(y_true, y_pred):
    ss_res = float(np.sum((y_true - y_pred) ** 2))
    ss_tot = float(np.sum((y_true - np.mean(y_true)) ** 2))
    return 1.0 - ss_res / ss_tot if ss_tot != 0 else float("nan")
```

8.2 k-ближайших соседей (лаб. 4, knn_wine.py)

```
def train_test_split(X, y, test_size=0.2, random_state=42):
    rng = np.random.RandomState(random_state)
    n = X.shape[0]
    indices = rng.permutation(n)
    n_test = int(round(n * test_size))
    return X[indices[n_test:]], X[indices[:n_test]], y[indices[n_test:]], y[indices[:n_test]]

class KNNClassifier:
    def __init__(self, k=3):
        self.k = k
    def fit(self, X, y):
        self.X_train = np.asarray(X, dtype=float)
        self.y_train = np.asarray(y)
    def predict(self, X):
        X = np.asarray(X, dtype=float)
        # ||a-b||^2 = ||a||^2 + ||b||^2 - 2 a*b
        a2 = np.sum(X ** 2, axis=1, keepdims=True)
        b2 = np.sum(self.X_train ** 2, axis=1, keepdims=True).T
        ab = X @ self.X_train.T
        dists = np.sqrt(np.maximum(a2 + b2 - 2 * ab, 0.0))
        k_nearest = np.argsort(dists, axis=1)[:, : self.k]
        k_labels = self.y_train[k_nearest]
        # majority vote among k neighbours
        predictions = np.array([
            np.bincount(row.astype(int)).argmax() for row in k_labels
        ])
        return predictions

def confusion_matrix(y_true, y_pred, classes):
    n = len(classes)
    cm = np.zeros((n, n), dtype=int)
    idx = {c: i for i, c in enumerate(classes)}
    for t, p in zip(y_true, y_pred):
        cm[idx[t], idx[p]] += 1
    return cm
```

8.3 Дерево решений (лаб. 5, lab5_tree.py)

```
def entropy(y):
```

```
    if len(y) == 0:
        return 0.0
    p = np.mean(y == 1)
    if p == 0.0 or p == 1.0:
        return 0.0
    return -p * np.log2(p) - (1.0 - p) * np.log2(1 - p)

def information_gain(col, y, values):
    total = len(y)
    h_parent = entropy(y)
    weighted = 0.0
    for v in values:
        mask = col == v
        w = np.sum(mask)
        if w == 0:
            continue
        weighted += (w / total) * entropy(y[mask])
    return h_parent - weighted

def build_tree(X, y, feature_indices, depth):
    node = Node()
    node.label = majority_label(y)
    node.pos_frac = np.mean(y == 1)
    # stopping conditions: pure node, max depth, features exhausted, few examples
    if (len(np.unique(y)) == 1 or depth >= MAX_DEPTH
            or len(feature_indices) == 0 or len(y) < 2):
        node.is_leaf = True
        return node
    best_gain = -1.0
    best_feat = None
    for fi in feature_indices:
        gain = information_gain(X[:, fi], y, np.unique(X[:, fi]))
        if gain > best_gain:
            best_gain = gain
            best_feat = fi
    if best_feat is None or best_gain <= 0.0:
        node.is_leaf = True
        return node
    node.feature_idx = best_feat
    remaining = [f for f in feature_indices if f != best_feat]
    for v in np.unique(X[:, best_feat]):
        mask = X[:, best_feat] == v
        node.children[v] = build_tree(X[mask], y[mask], remaining, depth + 1)
    return node

def predict_proba(node, x):
    # share of poisonous in the leaf the object falls into
    cur = node
    while not cur.is_leaf:
        v = x[cur.feature_idx]
        if v in cur.children:
            cur = cur.children[v]
        else:
            break
    return cur.pos_frac

def auc_roc(y_true, score):
    order = np.argsort(-score, kind="mergesort")
    y_sorted = y_true[order]
    P = np.sum(y_true == 1)
```

```
    Nn = np.sum(y_true == 0)
    tp = fp = 0
    points = [(0.0, 0.0)]
    for i in range(len(y_sorted)):
        if y_sorted[i] == 1:
            tp += 1
        else:
            fp += 1
        points.append((fp / Nn, tp / P))
    points.append((1.0, 1.0))
    # area by trapezoidal rule
    uniq = {}
    for fpr, tpr in points:
        uniq[fpr] = max(uniq.get(fpr, 0.0), tpr)
    xs = sorted(uniq.keys())
    auc = 0.0
    for i in range(1, len(xs)):
        auc += (xs[i] - xs[i - 1]) * (uniq[xs[i]] + uniq[xs[i - 1]]) / 2.0
    return auc, points
```

8.4 Логистическая регрессия (лаб. 6, lab6_logreg.py)

```
def sigmoid(z):
    return 1.0 / (1.0 + np.exp(-np.clip(z, -500, 500)))

def log_loss(y_true, p):
    eps = 1e-12
    p = np.clip(p, eps, 1.0 - eps)
    return -float(np.mean(y_true * np.log(p) + (1.0 - y_true) * np.log(1.0 - p)))

def gradient_descent(X, y, lr, n_iter):
    w = np.zeros((X.shape[1], 1))
    history = []
    for _ in range(n_iter):
        p = sigmoid(X @ w)
        grad = X.T @ (p - y) / X.shape[0]
        w = w - lr * grad
        history.append(log_loss(y, p))
    return w, history

def newton_method(X, y, n_iter):
    w = np.zeros((X.shape[1], 1))
    history = []
    for _ in range(n_iter):
        p = sigmoid(X @ w)
        grad = X.T @ (p - y) / X.shape[0]
        W = np.diag((p * (1.0 - p)).ravel())
        H = (X.T @ W @ X) / X.shape[0]
        step = np.linalg.solve(H, grad)
        w = w - step
        history.append(log_loss(y, p))
    return w, history

def predict(X, w):
    return (sigmoid(X @ w) >= 0.5).astype(int)

def classification_metrics(y_true, y_pred):
    tp = int(np.sum((y_pred == 1) & (y_true == 1)))
    tn = int(np.sum((y_pred == 0) & (y_true == 0)))
```

```
    fp = int(np.sum((y_pred == 1) & (y_true == 0)))
    fn = int(np.sum((y_pred == 0) & (y_true == 1)))
    acc = (tp + tn) / max(len(y_true), 1)
    prec = tp / max(tp + fp, 1)
    rec = tp / max(tp + fn, 1)
    f1 = 2.0 * prec * rec / max(prec + rec, 1e-12)
    return acc, prec, rec, f1, tp, tn, fp, fn
```